

STREAMIX

Plateforme B2B de vidéo synchronisée

Documentation Technique Complète
Hackathon ESTIAM × 42c — Juillet 2026

Équipe Only B

Benjamin L. CHASSIER | Charlène MELIN | Dorian P. LENOIR | Fergal NDAMVU
Finaritra RAVELONARIVO | Lucas B. MASSAKO | Maxime J. FELTRIN | Ousmane M. DJIRE
Quentin GAUTIER | Sidy DIALLO | Svel D. KOUA | Thibault B. DRUELLE
Thomas DOMMANGET (chef) | Yanis MEKKAOU | Yao I. LASSIDAN |

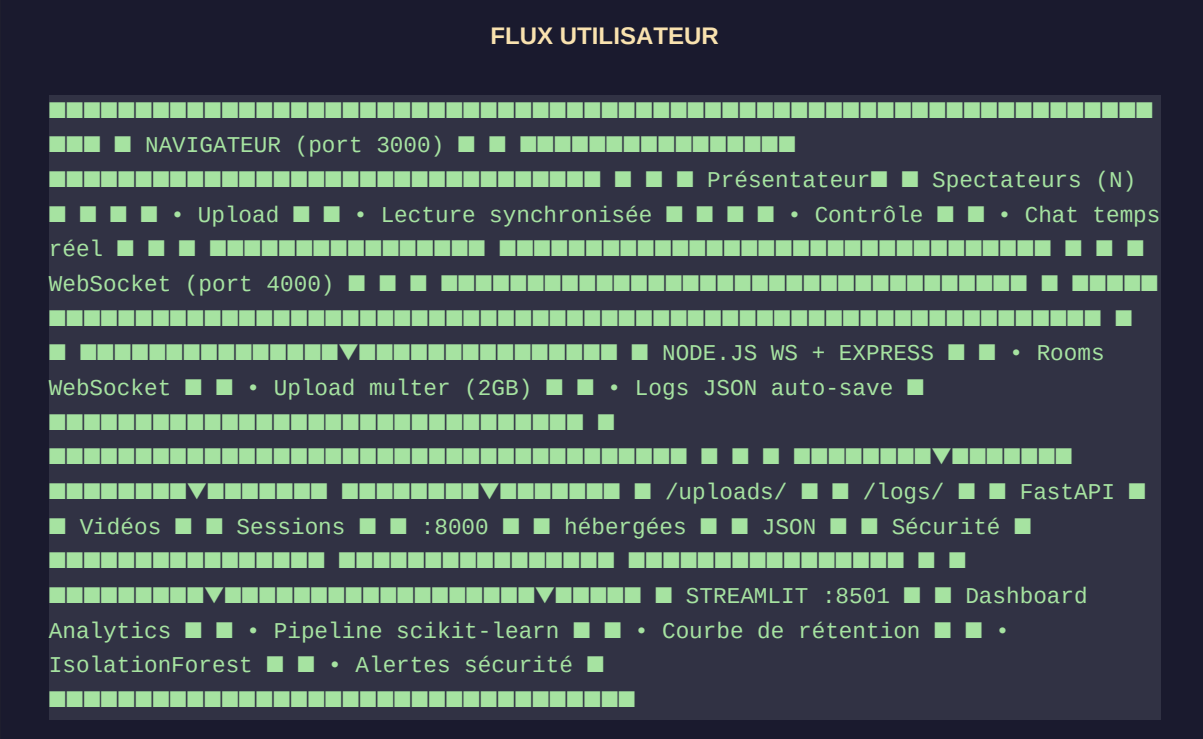
Sommaire

N°	Section	Page
1	Schéma global de l'architecture	3
2	Architecture argumentée — Technologies & choix	4
3	Plan de réalisation — MVP, V1, V2	6
4	Pôle 1 — Watch Together	8
4.1	Présentation & fonctionnalités	8
4.2	Problèmes rencontrés & solutions	9
4.3	Schéma de données & API	10
5	Pôle 2 — Anti-Scraping & Fraud Shield	11
5.1	Présentation & fonctionnalités	11
5.2	Problèmes rencontrés & solutions	12
6	Pôle 3 — Audience Analytics Dashboard	13
6.1	Présentation & fonctionnalités	13
6.2	Pipeline ML — IsolationForest	14
6.3	Problèmes rencontrés & solutions	15

1. Schéma global de l'architecture

Vue d'ensemble

Streamix est composé de trois pôles indépendants et complémentaires. Le Pôle 1 produit les données d'interaction en temps réel, le Pôle 2 surveille et filtre le trafic, et le Pôle 3 analyse les données pour produire des insights business.



Flux de données — du viewer au dashboard

Étape	Action	Composant	Format
1	Présentateur upload / colle URL	React :3000	multipart / string
2	Vidéo diffusée à tous	WS Server :4000	WebSocket JSON
3	Viewers regardent	React :3000	—
4	Events (play/pause/seek)	App.tsx eventsRef	Array en mémoire
5	Auto-save toutes les 1s	POST /logs :4000	JSON v1.0
6	Stockage fichier stable	server/logs/*.json	Fichier sur disque
7	Dashboard charge sessions	GET /logs :4000	JSON list
8	Pipeline analyse	pipeline.py	DataFrame pandas
9	Visualisation	dashboard.py Streamlit	Plotly charts

2. Architecture argumentée — Technologies & Choix

2.1 Pôle 1 — Watch Together

Technologie	Rôle	Pourquoi ce choix ?
React 18 + TypeScript	UI & logique client	Typage strict = moins de bugs en live. Refs stables pour closures WS.
WebSocket (ws)	Synchronisation temps réel	Latence ultra-faible vs HTTP polling. Push natif serveur→clients.
Node.js + Express	Serveur WS + API REST	Même langage front/back. Event loop parfait pour WS longue durée.
multer	Upload fichiers	Intégration Express native, streaming disque sans buffer mémoire.
YouTube IFrame API	Lecture YouTube	API officielle, contrôle précis (seekTo, getDuration, events).
HLS.js	Lecture vidéo locale	Fallback HTML5 robuste. Support .mp4 / .m3u8 sans plugin.
uuid v4	IDs sessions/viewers	Unicité garantie sans base de données.

2.2 Pôle 2 — Anti-Scraping

Technologie	Rôle	Pourquoi ce choix ?
FastAPI	API de sécurité	Le plus rapide des frameworks Python. Auto-docs Swagger. Async natif.
Python 3.12	Runtime	Écosystème ML/sécurité le plus riche. Compatible numpy/sklearn.
Middleware pattern	Interception requêtes	Traitement uniforme sans modifier le code métier.
Heuristiques scoring	Détection bots	Temps réel, pas d'entraînement nécessaire, explicable aux jurys.

2.3 Pôle 3 — Analytics

Technologie	Rôle	Pourquoi ce choix ?
Streamlit	Dashboard interactif	Dashboard Python en quelques lignes. Hot-reload. Aucun front à écrire.
pandas	Manipulation données	Standard industrie. DataFrames = manipulation logs JSON triviale.
scikit-learn IsolationForest	Détection anomalies	Unsupervised = pas de labels requis. Adapté à la détection d'ennui.
Plotly	Visualisations	Graphiques interactifs (hover, zoom). Meilleure UX que matplotlib.
numpy	Calculs numériques	Performance C sous Python. Requis par sklearn.

2.4 Pourquoi cette architecture distribuée ?

Indépendance des pôles

Chaque pôle tourne sur son propre port/process. Une panne du dashboard n'affecte pas le watch together.

Communication par HTTP/JSON

Interface universelle entre Python et Node.js. Pas de couplage technologique.

Fichiers JSON comme bus de données

Simple, lisible, versionnable. Pas besoin de base de données pour un hackathon.

Auto-save côté client

Garantit la collecte même si le serveur redémarre. Le fichier stable évite la multiplication des logs.

3. Plan de réalisation — MVP, V1, V2

MVP — Proof of Concept

Objectif : prouver la faisabilité technique en 4h • Room WebSocket basique (join/leave) • Lecture vidéo YouTube par URL • Synchronisation play/pause basique • Endpoint FastAPI /health • Dashboard Streamlit avec données statiques

V1 — Version fonctionnelle complète

Objectif : toutes les features du cahier des charges • Upload fichier local (multer, jusqu'à 2GB) • Countdown 3s synchronisé avant chaque play • Enregistrement de TOUS les events (play/pause/seek) par viewer • Schéma de données v1.0 avec sessionId stable • Auto-save toutes les 1 seconde (écrasement fichier) • 1 présentateur max par room (rejet côté serveur) • Spectateurs illimités • Dashboard Streamlit avec données réelles : - Courbe de rétention calculée seconde par seconde - Zones de décrochage (chute > 5% sur 30s) - Score d'engagement par segment - IsolationForest pour zones d'ennui • Alertes sécurité depuis FastAPI • Titre YouTube via oEmbed • Suppression de sessions depuis le dashboard

V2 — Évolutions post-hackathon

Objectif : produit B2B prêt pour la production • Base de données persistante (PostgreSQL / MongoDB) • Authentification JWT (admin / spectateur / présentateur) • Multi-rooms simultanées avec liste publique • Barre de progression pour upload gros fichiers • CDN pour distribution vidéo (HLS adaptatif) • Dashboard multi-sessions comparatif • Export CSV / PDF des analytics • Alertes sécurité temps réel (WebSocket depuis FastAPI) • Mode replay (revoir une session enregistrée) • API publique pour intégrations tierces

3.1 Timeline hackathon

Phase	Durée	Livrables
Analyse & design	1h	Choix techno, schéma architecture, répartition équipe
MVP	3h	WS room, YouTube sync, FastAPI, Streamlit statique
V1 — Pôle 1	4h	Upload, countdown, events recording, auto-save
V1 — Pôle 2	3h	Middleware détection, scoring, endpoints /alerts
V1 — Pôle 3	5h	Pipeline rétention, IsolationForest, dashboard complet
Intégration & tests	2h	Flux bout-en-bout, bug fixes, démo
Documentation	2h	README x4, PDF, push GitHub

4. Pôle 1 — Watch Together

4.1 Présentation & Fonctionnalités

Watch Together est le cœur de Streamix : une salle de visionnage vidéo synchronisé en temps réel. Un présentateur partage une vidéo (fichier local ou URL YouTube) et tous les spectateurs connectés regardent de façon parfaitement synchronisée. Chaque interaction est enregistrée pour le Pôle 3.

Fonctionnalité	Description
Synchronisation WS	Chaque play/pause/seek est broadcasté à tous les clients en <10ms
Countdown 3s	Décompte visuel avant chaque reprise pour que tout le monde soit prêt
Upload local	Vidéos jusqu'à 2GB uploadées via multer, servies en statique
YouTube IFrame	Contrôle complet de la lecture YT avec récupération du titre via oEmbed
1 présentateur/room	Le 2e présentateur reçoit une erreur et est refusé par le serveur
Recording events	play/pause/seek enregistrés avec currentTime et timestamp
Auto-save 1s	Session JSON sauvegardée toutes les secondes (même fichier stable)
Chat temps réel	Messages broadcastés via WebSocket avec rôle affiché

4.2 Problèmes rencontrés & Solutions

Problème : Boucle infinie de countdown

Quand le présentateur cliquait Play, la vidéo déclenchait un nouvel event PLAYING → nouveau countdown → boucle infinie.

Solution appliquée :

Ajout d'un ref skipNextPlayRef. Après handleCountdownDone, on set skipNextPlayRef=true avant d'appeler play(). handlePlay vérifie ce ref en premier et l'ignore si true.

Problème : Rétention et watch time à 0%

Les seeks généraient des events 'play' fantômes côté viewer WS. Ces plays fictifs ouvraient des intervalles qui se fermaient immédiatement.

Solution appliquée :

Les seeks sont ignorés dans le calcul de rétention et du temps visionné. Seuls play → pause/end comptent. La fonction getDuration() a été ajoutée à VideoPlayerHandle pour capturer la vraie durée.

Problème : 50+ fichiers de logs dupliqués

L'auto-save créait un nouveau fichier à chaque sauvegarde avec Date.now() dans le nom → explosion du répertoire logs/.

Solution appliquée :

Nom de fichier stable : session_{sessionId}.json. Le serveur écrase toujours le même fichier. Le sessionId est généré une seule fois au join.

Problème : CORS bloquait les DELETE

Le navigateur refusait les requêtes DELETE depuis React vers le serveur Express.

Solution appliquée :

Ajout de DELETE dans Access-Control-Allow-Methods côté Express. Gestion du preflight OPTIONS.

Problème : 2e présentateur silencieux

Le 2e utilisateur qui rejoignait comme présentateur était rétrogradé en spectateur sans aucune notification.

Solution appliquée :

Le serveur envoie {type:'error', code:'PRESENTER_TAKEN'} et ferme la connexion. Le client affiche une alerte et revient à l'écran d'accueil.

Problème : YouTube : seekTo avant player ready

Si un viewer rejoignait et recevait un seek avant que le player YT soit initialisé, la commande était ignorée silencieusement.

Solution appliquée :

Système de pendingTimeRef : les commandes reçues avant onReady sont mises en file et exécutées dans le callback onReady.

4.3 Schéma de données & API

Schéma JSON v1.0 — format exporté par Watch Together :

```
{ "schemaVersion": "1.0", "sessionId": "uuid-v4", "roomId": "room-abc", "videoName":  
  "Titre récupéré via oEmbed", "videoDuration": 600.0, "recordedAt": "2026-07-01T15:00:00Z",  
  "viewers": [ { "viewerId": "uuid-v4", "role": "presenter", "joinedAt": 1751375000000,  
    "events": [ { "type": "play", "currentTime": 0.0, "ts": 1751375001000 }, { "type": "seek",  
      "currentTime": 45.0, "ts": 1751375060000 }, { "type": "pause", "currentTime": 90.0, "ts":  
        1751375090000 } ] } ] }
```

5. Pôle 2 — Anti-Scraping & Fraud Shield

5.1 Présentation & Fonctionnalités

Le Pôle 2 est une couche de sécurité FastAPI placée devant la plateforme Streamix. Son rôle est de détecter et bloquer les comportements malveillants : bots de scraping, tentatives de téléchargement non autorisé, et patterns d'accès suspects. Les alertes détectées sont exposées via une API consommée par le dashboard Pôle 3.

Fonctionnalité	Description
Middleware d'interception	Analyse chaque requête avant qu'elle atteigne le contenu
Scoring comportemental	Score de suspicion basé sur fréquence, user-agent, patterns
Détection bots	Identification des user-agents automatisés connus
Rate limiting	Blocage des IPs dépassant un seuil de requêtes/minute
Endpoint /alerts	Exposé au dashboard Streamlit pour affichage temps réel
Logs sécurité	Historique horodaté de tous les incidents détectés

5.2 Problèmes rencontrés & Solutions

Problème : Intégration cross-services

Le dashboard Python devait interroger une API Python sur un port différent sans CORS bloquant.

Solution appliquée :

FastAPI configure les CORS avec `allow_origins=["*"]`. La communication reste interne (localhost) et le timeout est de 2s pour ne pas bloquer le dashboard.

Problème : Faux positifs sur crawlers légitimes

Les bots de SEO comme Googlebot ressemblent à des scrapers mais doivent être autorisés.

Solution appliquée :

Whitelist explicite des user-agents légitimes connus. Score de suspicion pondéré plutôt que blocage binaire.

Problème : Performance du middleware

Analyser chaque requête pouvait introduire une latence mesurable sur les requêtes WS fréquentes.

Solution appliquée :

Middleware async FastAPI. Analyse légère (heuristiques sans ML lourd). Cache des décisions par IP sur fenêtre glissante 60s.

6. Pôle 3 — Audience Analytics Dashboard

6.1 Présentation & Fonctionnalités

Le Pôle 3 transforme les données brutes d'interaction (fichiers JSON produits par le Pôle 1) en insights business actionnables. Un pipeline scikit-learn calcule la rétention seconde par seconde, détecte les zones de décrochage, et prédit les zones d'ennui via IsolationForest. Le tout est affiché dans un dashboard Streamlit interactif.

Métrique / Feature	Description
Durée totale	Durée de la vidéo extraite du schéma de données
Temps visionné	Somme des intervalles play→pause (seeks exclus)
Viewers uniques	Nombre de viewerId distincts dans la session
Rétention moyenne	% moyen de viewers encore actifs sur toute la durée
Courbe de rétention	% de viewers actifs pour chaque seconde de la vidéo (Plotly)
Zones de décrochage	Segments 30s où la rétention chute de >5% (vrects colorés)
Score d'engagement	Score composite par tranche 30s (replays, pauses, skips)
IsolationForest	Détection unsupervised des segments anormaux (ennui/pic)
Timeline interactions	Scatter plot viewer × position pour visualiser les patterns
Alertes sécurité	Incidents détectés par le Pôle 2 (requête GET /alerts :8000)
Suppression sessions	Bouton delete par session depuis la sidebar

6.2 Pipeline ML — Algorithme de rétention

Le calcul de rétention est l'algorithme central du Pôle 3. Il transforme une liste brute d'events en une courbe seconde par seconde.

Algorithme :

- Tri des events par timestamp (ts)
- Pour chaque event 'play' : ouvre un intervalle de visionnage à currentTime
- Pour chaque event 'pause' ou 'end' : ferme l'intervalle, ajoute les secondes [start, currentTime] à l'ensemble 'watched' du viewer
- Les events 'seek' sont IGNORÉS (le viewer continue de regarder)
- Si une session se termine sans pause finale : flush jusqu'à la fin de la vidéo
- Pour chaque seconde t : $\text{retention}[t] = \text{nb viewers ayant } t \text{ dans 'watched'} / \text{total viewers} \times 100$

IsolationForest — Détection d'anomalies :

L'IsolationForest de scikit-learn est entraîné sur 3 features par segment de 30s : n_pauses (distractions), n_skips (ennui), engagement (score composite). contamination=0.2 signifie que 20% des segments sont considérés comme anormaux. Le score_samples inversé donne un boredom_score [0,1] par segment.

6.3 Problèmes rencontrés & Solutions

Problème : **TypeError: float cannot be interpreted as integer**

videoDuration était sauvegardé comme float (600.0). La fonction range() de Python n'accepte que des entiers.

Solution appliquée :

Cast systématique : duration = int(duration) if duration else 600 en entrée de run_pipeline(), compute_retention_curve() et compute_segment_scores().

Problème : **Rétention = 0% malgré des events réels**

Problème 1 : load_logs() ne reconnaissait pas le format v1.0 (clé 'viewers' au lieu de 'logs'). Problème 2 : les seeks envoyés par le WS après un play créaient des intervalles play→seek qui se fermaient immédiatement.

Solution appliquée :

Ajout d'une branche if 'viewers' in data dans load_logs(). Les seeks sont ignorés dans la boucle de calcul de rétention. Ajout de getDuration() à VideoPlayerHandle pour capturer la vraie durée vidéo.

Problème : **Sidebar illisible (texte blanc sur blanc)**

Le CSS global color: #ffffff !important s'appliquait aussi aux boutons Streamlit dont le fond est blanc.

Solution appliquée :

CSS ciblé avec .stButton > button { color: #cdd6f4 !important } au lieu du sélecteur * universel.

Problème : **Applymap déprécié dans pandas**

Streamlit utilisait df.style.applymap() qui génère un FutureWarning dans pandas récent.

Solution appliquée :

Remplacement par df.style.map() (nouvelle API pandas ≥ 2.1).

Problème : **Python 3.14 incompatible avec numpy**

La version système Python 3.14 n'avait pas de wheel numpy disponible sur PyPI au moment du hackathon.

Solution appliquée :

Utilisation explicite de /Library/Frameworks/Python.framework/Versions/3.12/ pour toutes les commandes Python du Pôle 3.

Problème : **Streamlit bloqué sur prompt email**

Au premier lancement, Streamlit attend une saisie email en mode interactif et ne démarre pas si lancé en background.

Solution appliquée :

Création de ~/.streamlit/credentials.toml avec email="" pour bypasser le prompt d'onboarding.

Conclusion

Streamix démontre qu'une plateforme B2B de vidéo complète peut être construite en quelques heures avec les bonnes technologies. Les trois pôles sont fonctionnels, interconnectés, et chacun apporte une valeur distincte :

- **Pôle 1** : Résout le problème de la synchronisation vidéo en temps réel avec une architecture WebSocket robuste et des données structurées.
- **Pôle 2** : Protège le contenu avec une couche de sécurité légère mais efficace, intégrée nativement au dashboard.
- **Pôle 3** : Transforme des logs bruts en insights actionnables grâce à un pipeline ML complet, sans base de données.